

Bellerophon Language Documentation

Language Overview

Bellerophon is a domain-specific scripting language (DSL) for 3D printing. Write clean, procedural motions and parametric macros that compile directly to G-code for Klipper or Marlin firmware.

Note:

Philosophy: Precise, readable commands that make controlling 3D printers easier.

Getting Started & Platform Requirements

Bellerophon is a 3D printer scripting IDE and language designed for Klipper and Marlin macros. It allows you to write structured, parametric scripts and compile them into firmware-compatible G-code macros. This section describes system requirements, inputs, outputs, capabilities, and limitations.

1. Requirements

- **Software:** Java Runtime Environment (JRE) 19 or newer
- **Hardware:** Any computer capable of running Java

2. Running the IDE

Windows

1. Double-click `run.bat` to start the server
2. Your browser will open automatically to the Dimidium Ecosystem
3. Close the server window when finished

Linux / Mac

1. Install Java (OpenJDK 19 or newer)
2. Open a terminal in the SyntaxN folder
3. Run: `java -jar SyntaxN.jar`
4. Open browser to: `http://localhost:4567`
5. Press Ctrl+C in terminal to stop the server

Note:

The server runs locally on port 4567. No internet connection required after installation.

3. Compilation Targets: Klipper vs Marlin

The IDE can compile your .bph scripts to two different output formats:

- **Klipper** – Full support for all Bellerophon features, including `if` statements, advanced temperature commands, and Klipper-specific G-code macros.
- **Marlin** – A simpler output that omits features not available in Marlin firmware. **Conditional statements (`if/endif`) are not supported in Marlin mode**; all other movement, heating, looping, and macro commands work as expected (loops are expanded at compile time).

Select the target in the IDE before compiling. The generated .cfg (Klipper) or .gcode (Marlin) file will be saved accordingly.

4. Inputs

- Bellerophon accepts only .bph script files.
- The IDE provides real-time syntax validation, highlighting errors as you type.
- Scripts are line-oriented: each command must be on its own line; blank lines between commands are ignored.
- Supported commands include: movement, temperature control, heaters, fans, waits, pauses, macros calls, repeat loops, Brepeat loops, conditional execution (Klipper only), extruder control, and bed/probe operations.

5. Outputs

- Compiles .bph scripts into either Klipper macro .cfg files or Marlin .gcode files.
- Compiled macros can be deployed to printers via **Gravity Hub** or directly from Mainsail (Klipper) or any Marlin host.
- The IDE provides Build Log and Exceptions tabs for debugging and catching compilation errors.

6. Capabilities

- Compile structured macros for Klipper or Marlin with correct syntax.
- Support for arithmetic expressions in coordinates and parameters – **globally available**.
- Mathematical functions: `sin`, `cos`, `tan`, `sqrt`, `abs` (absolute value), `sign` (signum), power operator `^`, and constant `pi`.
- **Local user variables** – define and reuse variables inside macros.
- Parametric loops using `repeat` (static repetition) and `Brepeat` (dynamic, iteration-aware loops), including support for **nested loops** and iterator-based expressions.
- **Parametric Geometry**: Mathematical functions allow the generation of procedural motion paths such as circles, spirals, or arbitrary curves.
- Conditional execution with `if` statements (Klipper only), allowing decisions based on temperatures or other parameters.
- Temperature management for extruder, bed, and chamber (`Set_Heater_Temperature`, `Heat`, `WaitForTemp`).
- Printer movement commands, including absolute and relative positioning, X/Y/Z moves, extruder control, and dwell/wait commands.
- Macro calls within macros (`M.call`), allowing reuse and modular scripts.
- Bed and probe management: `BED_MESH_CALIBRATE`, `PROBE_CALIBRATE`, `LoadBedMesh`.
- Fan control, pressure advance, speed settings, and idle timeout (`TIMEOUT_SET`).
- Pause, resume, cooldown, and SD card file printing (`PRINTFILE`).

- **IDE Reference Panel:** Displays all Bellerophon tokens in a user-friendly format. Users can pin commands to a sidebar for quick access while typing, reducing errors and speeding up workflow.
- **Session Restoration:** The IDE automatically restores the previous editing session to prevent accidental data loss and allow users to continue work seamlessly.
- **Dimidium Ecosystem Integration:** Compiled macros can be deployed and managed through the Dimidium Gravity Hub dashboard or executed directly within Mainsail (Klipper) or any Marlin host.
- **Printer Profile UI:** The IDE provides a graphical interface to set hardware limits and default printing parameters (nozzle diameter, filament diameter, layer height, extrusion multiplier). These settings are used as defaults unless overridden by macro commands.

7. Limitations

- Cannot slice 3D models; only generates G-code from Bellerophon `.bph` scripts.
- Does not modify existing G-code files automatically.
- Some advanced or rare G-code commands may not be supported.
- **Variables are local to macros** – they do not persist across macros (global variables are a planned future feature).
- **Marlin target:** Conditional statements (`if/endif`) are not supported and will be ignored.

8. Workflow Diagram

High-level workflow of using Bellerophon:

`.bph` script → IDE (edit & validate) → Select target → Compile → `.cfg` (Klipper) / `.gcode` (Marlin) → Deploy → Run macros

Program Structure

A Bellerophon program consists of one or more macros. Each macro starts with `M.title` and ends with `M.end`.

```
M.title "Macro Name"

    [statement 1]

    [statement 2]

    [statement 3]

M.end
```

Each command must be on its own line. Indentation is optional but recommended. Comments start with `#`.

Local User Variables

Bellerophon supports **local variables** defined inside macros. Variables are scoped to the macro they are defined in and are not accessible outside it.

Note:

Scope: Variables are local to the macro. They do not persist across macro calls. Global variables are a planned future feature.

Assignment Syntax

To define or update a variable, use the assignment statement:

```
variable_name = expression
```

Supported Expressions

Variables can be assigned any valid expression:

- Numbers: 150, 3.14, -5
- Arithmetic: +, -, *, /, ^ (power)
- Parentheses: (base + offset) * 2
- Functions: sin(), cos(), tan(), sqrt(), abs(), sign()
- Constant: pi
- Variable references: my_x, speed (must be defined earlier in the macro)

Example

```
M.title "example"

  base = 100

  offset = 50

  speed = (base + offset) * 2

  SetSpeed = speed

  MoveTo x=base y=offset

  MoveTo x=base+offset y=base-offset

M.end
```

Output

```
[gcode_macro example]

gcode:

  G1 F300
```

G1 X100.000 Y50.000

G1 X150.000 Y50.000

Statement Reference

COMMAND	SYNTAX	EXAMPLE	G-CODE / NOTES
Assignment	variable = expression	my_x = 150	Stores the expression result in a local variable. Variables are scoped to the macro.
Home	Home Or Home -> coordList	Home Or Home -> x=0 y=0 z=0	G28 / G28 X Y Z
Move	Move direction	Move left / right / center / up / down	G1 with directional offset. Iterates 1 mm. center defaults to X0 Y0
MoveTo	MoveTo coordList	MoveTo x=100 y=100 z=0.2	G1 X Y Z E
Absolute	Absolute	Absolute	G90
Relative	Relative	Relative	G91
Heat	Heat target=number	Heat extruder=200	Sets temperature and waits until it is reached . Extruder → M109, Bed → M190, Chamber → M141
Set_Heater_Temperature	Set_Heater_Temperature target=number	Set_Heater_Temperature bed=60	Sets temperature but does NOT wait . Extruder → M104, Bed → M140, Chamber → M141
WaitForTemp	WaitForTemp target	WaitForTemp extruder	Waits until the target reaches its current set temperature. Does not change the temperature. Extruder → M109, Bed → M190, Chamber → M141
Cooldown	Cooldown target (optional)	Cooldown extruder or just Cooldown	M104 S0 / M140 S0 / TURN_OFF_HEATERS. If target omitted, turns off all heaters.
TimeoutSet	TIMEOUT_SET seconds	TIMEOUT_SET 300	Set idle timeout (seconds). Maps to Klipper's SET_IDLE_TIMEOUT TIMEOUT=300 (ignored in Marlin).
Pause	Pause	Pause	PAUSE (Klipper) / M0 (Marlin)
Resume	Resume	Resume	RESUME (Klipper) / M108 (Marlin)
Respond	Respond MSG "text"	Respond MSG "Print complete"	RESPOND MSG="text" (Klipper) / M117 (Marlin)

COMMAND	SYNTAX	EXAMPLE	G-CODE / NOTES
<code>SetSpeed</code>	<code>SetSpeed = number</code>	<code>SetSpeed = 100</code>	G1 F#
<code>SetFan</code>	<code>SetFan = number</code>	<code>SetFan = 255</code>	SET_FAN SPEED=# (Klipper) / M106 S# (Marlin)
<code>SetNozzle</code>	<code>SetNozzle = number</code>	<code>SetNozzle = 0.6</code>	Sets the nozzle diameter (mm) for extrusion calculations. Modifies PrinterSettings for the current macro. Default is taken from the printer profile UI.
<code>SetFilament</code>	<code>SetFilament = number</code>	<code>SetFilament = 2.85</code>	Sets the filament diameter (mm) for extrusion calculations. Modifies PrinterSettings for the current macro. Default is taken from the printer profile UI.
<code>SetLayerHeight</code>	<code>SetLayerHeight = number</code>	<code>SetLayerHeight = 0.3</code>	Sets the layer height (mm) for extrusion calculations and the Layer command. Modifies PrinterSettings for the current macro. Default is taken from the printer profile UI.
<code>SetExtrusionMultiplier</code>	<code>SetExtrusionMultiplier = number</code>	<code>SetExtrusionMultiplier = 0.95</code>	Adjusts the extrusion multiplier (flow rate compensation). Modifies PrinterSettings for the current macro. Default is taken from the printer profile UI.
<code>EnableAutoExtrude</code>	<code>EnableAutoExtrude = 1 0</code>	<code>EnableAutoExtrude = 1</code>	Enables (1) or disables (0) automatic extrusion calculation for MoveTo commands. Manual e=... overrides auto-extrusion.
<code>Layer</code>	<code>Layer number ... end</code>	<code>Layer 5 ... end</code>	Repeats the enclosed block for the specified number of layers, automatically raising Z by the current LayerHeight after each layer.
<code>PrintFile</code>	<code>PRINTFILE "file.gcode"</code>	<code>PRINTFILE "cube.gcode"</code>	SDCARD_PRINT_FILE FILENAME="file.gcode" (Klipper) / M23 (Marlin)
<code>Call Macro</code>	<code>M.call "MacroName"</code>	<code>M.call "CalibrateBed"</code>	Calls another macro (Klipper) / M98 (Marlin)
<code>If / EndIf</code>	<code>if condition ... endif</code>	<code>if extruder > 190 ... endif</code>	Klipper only ; compiled to Jinja condition. Not available in Marlin target.
<code>RelativeExtrusion</code>	<code>RelativeExtrusion</code>	<code>RelativeExtrusion</code>	M83
<code>ResetExtruder</code>	<code>ResetExtruder</code>	<code>ResetExtruder</code>	G92 E0
<code>BedMeshCalibrate</code>	<code>BedMeshCalibrate</code>	<code>BedMeshCalibrate</code>	BED_MESH_CALIBRATE (Klipper) / G29 (Marlin)
<code>Level</code>	<code>Level</code>	<code>Level</code>	Alias for BedMeshCalibrate → triggers BED_MESH_CALIBRATE or G29
<code>ProbeCalibrate</code>	<code>ProbeCalibrate</code>	<code>ProbeCalibrate</code>	PROBE_CALIBRATE (Klipper) – not available in Marlin

COMMAND	SYNTAX	EXAMPLE	G-CODE / NOTES
LoadBedMesh	LoadBedMesh "profile"	LoadBedMesh "default"	BED_MESH_PROFILE LOAD=profile (Klipper) – not available in Marlin
SetPressureAdvance	SetPressureAdvance number	SetPressureAdvance 0.04	SET_PRESSURE_ADVANCE ADVANCE=# (Klipper only)
Dwell	Dwell time S ms (optional unit)	Dwell 0.5 S or Dwell 500 ms	G4 P# (P is milliseconds). If unit omitted, defaults to seconds. Example: Dwell 0.5 → G4 P500

Loops & Arithmetic Operators

Bellerophon supports two types of loops: `repeat` for static repetition and `Brepeat` for dynamic loops.

Note:

Positioning Context in Loops: The behavior of movement inside a loop depends heavily on the active mode:

- **Relative (G91):** Coordinates are calculated as offsets from the last position. Each move builds on the previous one. Best for iteratively forming shapes, spirals, zig-zags, or any flowing patterns.
- **Absolute (G90):** Coordinates are calculated from a fixed origin on the bed. Each move goes to a specific point. Use this only when you need precise positions that don't depend on the previous move, like moving to a center point or fixed markers.

Example: Absolute vs Relative in Loops

The same loop code produces very different results depending on the active mode:

```
# Absolute Mode Example

Absolute

MoveTo x=10

Brepeat 4
    MoveTo x+=5
end
```

```
# Relative Mode Example

Relative

MoveTo x=10

Brepeat 4
    MoveTo x+=5
```

end

Output:

Absolute Mode Output

```
[gcode_macro Absolute!-tester]
```

```
gcode:
```

```
G90
```

```
G1 X10.000
```

```
G1 X15.000
```

```
G1 X20.000
```

```
G1 X25.000
```

```
G1 X30.000
```

Relative Mode Output

```
[gcode_macro Relative!-tester]
```

```
gcode:
```

```
G91
```

```
G1 X10.000
```

```
G1 X5.000
```

```
G1 X5.000
```

```
G1 X5.000
```

```
G1 X5.000
```

Note:

Key observation: In absolute mode, `x+=5` calculates a new absolute position (10+5=15, then 15+5=20, etc.). In relative mode, it adds 5 to the current position each time, creating equal-sized steps from wherever the previous move ended.

Assignment Operators

Operators allow for precise control during movement:

-  `=` Direct Assignment
-  `+=` Add to current
-  `-=` Subtract from current
-  `*=` Multiply current
-  `/=` Divide current

1. Static Repeat (`repeat`)

The `repeat` command simply duplicates the enclosed G-code commands a specified number of times during compilation.

```
repeat 3

    MoveTo x+=10 y+=10

    Respond MSG "Moving..."

end
```

Output (repeat):

```
G1 X10.0 Y10.0

RESPOND MSG="Moving..."

G1 X20.0 Y20.0

RESPOND MSG="Moving..."

G1 X30.0 Y30.0

RESPOND MSG="Moving..."
```

Note:

The commands are physically duplicated in the output G-code. Great for simple, fixed repetitions.

2. Dynamic Loop (`Brepeat`)

The `Brepeat` command creates a loop that supports arithmetic expressions and mathematical functions. Unlike `repeat`, this loop exposes an iterator variable `i` which increments each iteration starting from 0. Expressions are evaluated during compilation, producing numeric G-code values.

Note:

Important: The iterator `i` is only valid inside `Brepeat` loops, but **mathematical functions (sin, cos, tan, sqrt, abs, sign) and the power operator (^) are globally available anywhere an expression is allowed** (e.g., in `MoveTo`, `Heat`, `SetSpeed`, etc.).

Example: Using the iterator

```
Brepeat 5

    MoveTo x=i

end
```

Example: Mathematical Expressions (global availability)

Beleroophon supports arithmetic expressions anywhere: including exponentiation `^`, absolute value `abs()`, signum `sign()`, and the constant `pi`.

```
Repeat 3

  MoveTo x=i^2 * 5 + abs(i-2)

end

Heat extruder=180 + 20*sin(3.14159/2)

# works outside loops too
```

Example: Drawing a Square (Relative)

Using **Relative** mode ensures each move is from the previous point, naturally forming shapes when iterating.

```
Relative

Repeat 4

  MoveTo x=10 y=0

  MoveTo x=0 y=10

  MoveTo x=-10 y=0

  MoveTo x=0 y=-10

end
```

Example: Drawing a Circle (Relative)

Using **Relative** mode allows the circle to be built incrementally, keeping each move relative to the last point for smooth motion.

```
Relative

Repeat 60

  MoveTo x=cos(i*0.1)*5 y=sin(i*0.1)*5

end
```

Example: Drawing a Spiral (Relative)

Using **Relative** mode allows complex parametric shapes to be built incrementally. This spiral uses the loop counter `i` to calculate both angle and radius, creating an expanding spiral pattern.

```
# Spiral test macro

M.title "spiral_test"

MoveTo x = 160 y = 160 z= 10

Relative
```

```

SetSpeed = 6500

Repeat 65

    MoveTo x=cos(i*10)*(i*0.1) y=sin(i*10)*(i*0.1)

end

Dwell 2 ms

Absolute

MoveTo x = 150 y = 150

Dwell 2 s

Home

M.end

```

Note:

How the spiral formula works: The expression `cos(i*10)*(i*0.1)` and `sin(i*10)*(i*0.1)` create an expanding spiral where:

- `i*10` controls the angular step (angle increases quickly, making many turns)
- `i*0.1` controls the radius growth (spiral expands outward slowly)
- The product creates a spiral that grows in radius as it rotates
- Using **Relative** mode means each move is an offset from the previous position

Example Program

```

M.title "Bed Leveling Routine"

Absolute

SetSpeed = 3000

Home -> x=0 y=0

MoveTo z=5

Heat bed=60

Respond MSG "Bed heating complete"

Level

LoadBedMesh "default"

MoveTo x=100 y=100

ProbeCalibrate

SetPressureAdvance 0.04

ResetExtruder

```

RelativeExtrusion

Move right

Move up

Pause

Resume

Cooldown

M.end